DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Declarative pipelines

Section 1. Streaming tables, materialized views, and the refresh that destroys data.

SECTION 1 of 10 WEIGHT 22% TOPIC Pipelines

admin@sthithapragna.com

You declare **what the table should contain**. The engine works out how to keep it that way.

Three dataset types

TYPE	WHAT IT IS
Streaming table	A Unity Catalog managed table that is a streaming target. Each row processed once.
Materialized view	A managed table holding pre-computed results, kept current by incremental refresh.
Temporary view	Pipeline-scoped. Organises logic without materialising anything to storage.

ON THE EXAM

Both streaming tables and materialized views are Unity Catalog managed tables. The difference is how they are kept up to date, not what they are.

Streaming table against materialized view

STREAMING TABLE

Process each row once

- Right for ingestion and low latency
- Append-only and high-volume workloads
- Event-driven, from storage or a message bus
- Cost scales with new data, not total data

MATERIALIZED VIEW

Keep a result correct

- Right for complex transformations and analytics
- Handles joins, aggregations, late updates
- Results pre-computed, so queries are fast
- You cannot modify the data directly

THE TIEBREAKER

Ask whether the source is an unbounded stream of new records, or a query whose answer must stay correct as inputs change.

What 'processed once' buys and costs

- **Buys:** the cost of an update depends on how much arrived, not how much you hold.
- Ideal for bronze ingestion. Databricks recommends streaming tables for **most ingestion cases**.
- **Costs:** a row already processed is not revisited. Late corrections to old rows are not naturally handled.
- Changing stateful logic invalidates existing state, which forces a full refresh.

> ON THE EXAM
>
> For files landing in cloud object storage, Auto Loader gives incremental and idempotent loading into a streaming table.

Correct by construction

- The **query definition controls the output**. You cannot write into a materialized view directly.
- The engine performs **incremental refresh** where it can, processing only what changed.
- Handles updates and deletes in the source naturally, because it recomputes the answer.
- When incremental refresh is not possible, it recomputes, and that cost scales with total data.

> ON THE EXAM
>
> Use them for silver logic with enrichment joins, and for gold aggregations, metrics and summaries.

Which type for which layer

LAYER	USUAL CHOICE
Bronze	Streaming table. Ingest once, append, keep the raw record.
Silver	Streaming table for cleansing. Materialized view for enrichment joins.
Gold	Materialized view. Pre-computed aggregations, metrics and summaries.

ON THE EXAM

Silver is the layer where the answer genuinely depends on the work. Joins against dimensions favour a materialized view; row-level cleansing does not.

The difference is one word

- A **batch read** in the function body produces a materialized view.
- A **streaming read** in the same body produces a streaming table.
- In SQL: **CREATE MATERIALIZED VIEW** against **CREATE STREAMING TABLE**.
- Not every source supports a streaming read. Some sources should always be read as a stream.

> ON THE EXAM
>
> Pipeline dataset functions must return a DataFrame, and must not have side effects. Side effects inside them cause unpredictable behaviour.

The Python API was renamed after the exam guide

YOU WILL SEE	CURRENT FORM
`import dlt`	from pyspark import pipelines as dp
`@dlt.table with a streaming read`	@dp.table
`@dlt.table with a batch read`	@dp.materialized_view
`@dlt.view`	@dp.temporary_view
`dlt.apply_changes(...)`	dp.create_auto_cdc_flow(...)
`@dlt.expect(...)`	dp.expect(...)

ON THE EXAM

Documented on 10 July 2026, one week after the exam guide was published. The older module still works, and SQL did not change at all.

SQL did not change, and neither did the concepts

- **CREATE STREAMING TABLE**, **CREATE MATERIALIZED VIEW** and **CREATE FLOW** are identical in both.
- If you are unsure which Python form an exam question expects, the concepts are what is really being tested.
- Some DLT naming survives on purpose: classic pipeline billing codes and event log schemas still say DLT.
- Apache Spark 4.1 includes declarative pipelines, but **AUTO CDC, expectations and temporary views are Databricks-only**.

> ON THE EXAM
>
> The exam tests which dataset type suits a workload far more than it tests which decorator spells it.

How data actually gets into a dataset

FLOW	WHAT IT DOES
Append flow	The default for both dataset types. Adds new records to the target.
AUTO CDC flow	Streaming flow handling out-of-order changes. SCD Type 1 and Type 2.
Materialized view flow	A batch flow that processes only new data and changes where it can.
REPLACE WHERE flow	Incremental batch processing of joins and aggregations.

ON THE EXAM

A flow is usually created implicitly by your query. You define one explicitly when several sources must write into one target.

Append flows avoid a full refresh

- An append flow adds a **new streaming source** to an existing streaming table without a full refresh.
- It is also how you **backfill** historical data into a streaming table that is already live.
- And how you combine several sources into one target without a union in a single query.
- Without it, adding a source to a streaming table generally means reprocessing everything.

> ON THE EXAM
>
> If a question needs a new source or a backfill added to a live streaming table without disruption, this is the mechanism.

Triggered against continuous

TRIGGERED

Refresh, then stop

- Processes data available when the update began
- Compute runs only long enough to finish
- Cheaper, and the usual default
- New data waits for the next trigger

CONTINUOUS

Keep it fresh

- Processes new data as it arrives
- Requires an always-running cluster
- More expensive, lower latency
- For millisecond latency, use real-time mode

COMMONLY CONFUSED

Pipeline mode is independent of dataset type. Both materialized views and streaming tables update in either mode.

Use a continuous job, not continuous mode

- Databricks recommends running a continuous pipeline via a **continuous job**, rather than setting pipeline mode to continuous.
- The job then manages the execution lifecycle, and **unlocks serverless performance modes** that built-in continuous mode does not support.
- Trigger interval is best set **per table**, because streaming and batch queries have different defaults.
- Standalone materialized views and streaming tables **always** run triggered, whatever orchestrates them.

> ON THE EXAM
>
> This is a recent recommendation and older material contradicts it. Job orchestration is now the preferred route to continuous execution.

What a full refresh does to a streaming table

- It **truncates the table, clears the checkpoints**, and reprocesses all source data from the beginning.
- If the source no longer holds that data, it **cannot be recovered**. That is data loss.
- Downstream dependent tables must be fully refreshed too, or the pipeline fails.
- Block it with the table property **pipelines.reset.allowed** set to false.

> ON THE EXAM
>
> A Kafka source with 24-hour retention plus a full refresh run after that window equals messages that no longer exist anywhere.

Materialized views are not affected this way

- A materialized view returns the **same result** whether refreshed normally or fully.
- It is recomputing an answer from sources, so there is no checkpoint state to destroy.
- A streaming table resets all state and checkpoint information, and can drop records.
- Full refreshes must be triggered **explicitly**. Nothing runs one by accident.

> ON THE EXAM
>
> Same words, entirely different consequences. Which dataset type is being refreshed changes whether the operation is safe.

Legitimate reasons for a full refresh

- You changed **stateful logic**, such as a watermark threshold or the aggregation columns.
- The streaming checkpoint has been corrupted or deleted.
- Schema tracking or schema location files have been lost.
- Previously processed data would now be inconsistent with updated logic or schema.

> ON THE EXAM
>
> Streaming state is built incrementally. Change the logic and the existing state no longer matches it, so it has to be rebuilt.

Standalone against a full pipeline

STANDALONE

One dataset, managed for you

- SQL only, from a warehouse or serverless notebook
- Databricks manages the pipeline behind it
- Always triggered. Schedule or trigger on update
- Shows as MV/ST on the jobs page

LAKEFLOW PIPELINE

Many datasets, authored as a unit

- SQL and Python, with dependency orchestration
- Sinks, private datasets, snapshot CDC
- Triggered or continuous
- Shows as ETL on the jobs page

ONE-WAY DOOR

A standalone dataset cannot be moved into a pipeline later. It has to be recreated there.

What to carry into the exam

- Streaming table processes each row **once**. Materialized view keeps a query's **answer correct**.
- Batch read gives a materialized view, streaming read gives a streaming table.
- Full refresh on a streaming table **clears checkpoints and can lose data**. On a materialized view it is harmless.
- Append flows add sources or backfill **without** a full refresh.
- Mode is independent of dataset type. Triggered is cheaper, continuous is fresher.

> ON THE EXAM
>
> When a question describes late-arriving corrections to already-processed rows, a plain streaming table is usually the wrong answer.

A pipeline ingests clickstream events from a message bus into a streaming table. The bus retains messages for 24 hours. The table holds 18 months of history and feeds three downstream materialized views.

An engineer changes the watermark threshold on a stateful aggregation and is told the pipeline requires a full refresh to apply it.

What is the most important consequence to raise?

A The refresh will be slow, but all 18 months will be rebuilt correctly from the message bus.

B Only data still within the 24-hour retention window can be reprocessed; the rest is lost.

C The materialized views will need rebuilding, but the streaming table itself is unaffected.

D Full refresh applies only to materialized views, so the watermark change will not take effect.

◆ PAUSE AND DECIDE

What is the most important consequence to raise?

A. The refresh will be slow, but all 18 months will be rebuilt correctly from the message bus.

The bus holds 24 hours. It cannot supply 18 months, however long you wait.

B. Only data still within the 24-hour retention window can be reprocessed; the rest is lost.

Full refresh truncates the table and clears checkpoints, then replays from source. The source no longer has it.

C. The materialized views will need rebuilding, but the streaming table itself is unaffected.

Backwards. The streaming table is truncated, and downstream tables must be refreshed too or the pipeline fails.

D. Full refresh applies only to materialized views, so the watermark change will not take effect.

It applies to both, and changing stateful logic is a documented reason to need one.

ON THE EXAM

Retention on the source versus history in the table is the pairing to spot. When the table holds more than the source can replay, a full refresh destroys the difference.

A silver dataset joins order records against a customer dimension table. Customer records are updated regularly, including corrections to addresses on customers who ordered months ago.

The team implemented this as a streaming table and reports that corrected customer details never appear for historical orders.

What is the correct fix?

A Add an append flow so the corrections are appended to the streaming table.

B Switch the dataset to a materialized view, so the join result is recomputed as sources change.

C Run the pipeline in continuous mode so corrections are picked up as they arrive.

D Shorten the trigger interval so the streaming table updates more frequently.

◆ PAUSE AND DECIDE

What is the correct fix?

A. Add an append flow so the corrections are appended to the streaming table.

Appends new rows. It does not revise rows already written.

B. Switch the dataset to a materialized view, so the join result is recomputed as sources change.

An enrichment join where old rows must change is exactly what materialized views are for.

C. Run the pipeline in continuous mode so corrections are picked up as they arrive.

Changes latency, not semantics. Rows already processed are still not revisited.

D. Shorten the trigger interval so the streaming table updates more frequently.

Same mistake as C. More frequent updates to a process-once table still never revisit old rows.

ON THE EXAM

Process-once is a semantic property, not a timing one. No scheduling change makes a streaming table revisit a row it has already handled.

A live streaming table ingests transactions from one regional system. A second region is being onboarded, and its historical transactions must also be loaded into the same table.

The table has 400 million rows and downstream gold materialized views that must stay available throughout.

Which approach adds the second region with least disruption?

A Rewrite the query as a union of both sources and run a full refresh.

B Define an append flow for the second region's source, and another to backfill its history.

C Create a second streaming table and a materialized view that unions the two.

D Switch the table to continuous mode so it picks up the new source automatically.

◆ PAUSE AND DECIDE

Which approach adds the second region with least disruption?

A Rewrite the query as a union of both sources and run a full refresh.

Works, and reprocesses 400 million rows while clearing checkpoints. The expensive route.

B Define an append flow for the second region's source, and another to backfill its history.

Append flows exist to add sources and backfill history to a live streaming table without a full refresh.

C Create a second streaming table and a materialized view that unions the two.

Workable but adds a dataset and a layer, and changes what downstream views read from.

D Switch the table to continuous mode so it picks up the new source automatically.

Mode controls when work happens, not which sources feed a table.

ON THE EXAM

Adding a source, or backfilling, without a full refresh is the exact purpose of append flows. Recognise the requirement wording.

> An analyst needs a single pre-aggregated table to accelerate a dashboard. They work entirely in the SQL editor against a serverless warehouse, want it refreshed on a schedule, and need no orchestration or intermediate datasets.
>
> A colleague suggests building a full pipeline with Python so it can be extended later.

What is the better recommendation?

A. Build the full pipeline now, since a standalone dataset can be moved into one later if needed.

B. Create a standalone materialized view with a schedule, accepting it must be recreated if a pipeline is needed later.

C. Create a standalone streaming table, since scheduled refresh requires streaming semantics.

D. Build the full pipeline, because standalone datasets cannot be refreshed on a schedule.

◆ PAUSE AND DECIDE

What is the better recommendation?

A Build the full pipeline now, since a standalone dataset can be moved into one later if needed.

The reasoning is wrong. Standalone datasets cannot be moved into a pipeline; they must be recreated.

B Create a standalone materialized view with a schedule, accepting it must be recreated if a pipeline is needed later.

One dataset, SQL, warehouse, scheduled refresh, no pipeline-only features. Exactly the standalone case.

C Create a standalone streaming table, since scheduled refresh requires streaming semantics.

Invented. A pre-aggregated dashboard table is a materialized view.

D Build the full pipeline, because standalone datasets cannot be refreshed on a schedule.

They can, with SCHEDULE, TRIGGER ON UPDATE, or a SQL task in a job.

ON THE EXAM

Note that A reaches the wrong conclusion via a true-sounding claim. The one-way door is real, but it argues for honesty about the trade-off, not for over-building.

A team runs a pipeline in continuous mode to keep a gold layer fresh for an operations dashboard. Costs are higher than expected and they are told serverless performance modes could reduce them.

They confirm the freshness requirement is genuine and cannot be relaxed to a triggered schedule.

What should they change?

A Nothing. Continuous mode is the only way to achieve continuous freshness.

B Orchestrate the pipeline with a continuous job instead of setting pipeline mode to continuous.

C Convert the gold materialized views to streaming tables, which are cheaper in continuous mode.

D Set a short trigger interval on the pipeline and switch to triggered mode.

◆ PAUSE AND DECIDE

What should they change?

A Nothing. Continuous mode is the only way to achieve continuous freshness.

A continuous job achieves the same freshness and manages the lifecycle.

B Orchestrate the pipeline with a continuous job instead of setting pipeline mode to continuous.

Databricks' current recommendation. Job orchestration unlocks serverless performance modes built-in continuous mode does not support.

C Convert the gold materialized views to streaming tables, which are cheaper in continuous mode.

Mode is independent of dataset type, and gold aggregations are materialized view work.

D Set a short trigger interval on the pipeline and switch to triggered mode.

Relaxes the freshness requirement the question said was fixed.

ON THE EXAM

A recent recommendation that older material contradicts. Job orchestration is now the preferred route to continuous execution.

A bronze streaming table ingests files landing in cloud storage. A junior engineer has twice run a full refresh on it while debugging, each time forcing an expensive reprocess of the entire history and breaking three downstream tables.

The team wants to prevent this while keeping the ability to perform a full refresh deliberately during a planned migration.

What is the most appropriate control?

A. Set the table property pipelines.reset.allowed to false, and change it during the planned migration.

B. Convert the bronze table to a materialized view so full refresh becomes harmless.

C. Remove the junior engineer's permission on the pipeline entirely.

D. Enable skipChangeCommits on the downstream tables so they survive a full refresh.

◆ PAUSE AND DECIDE

What is the most appropriate control?

A Set the table property pipelines.reset.allowed to false, and change it during the planned migration.

Purpose-built for this. It blocks accidental refreshes and can be lifted deliberately when one is genuinely wanted.

B Convert the bronze table to a materialized view so full refresh becomes harmless.

Bronze ingestion is streaming table work. Changing the dataset type to dodge an operation is the wrong trade.

C Remove the junior engineer's permission on the pipeline entirely.

Addresses one person, not the risk. Anyone else can still do it tomorrow.

D Enable skipChangeCommits on the downstream tables so they survive a full refresh.

Helps downstream tables tolerate it, and does nothing about the bronze table being truncated.

ON THE EXAM

Both requirements matter: prevent the accident, and keep the deliberate operation available. Only a reversible control does both.